Trefzer

TRANSAKTIONEN, ACID, CAP

HERZLICH WILLKOMMEN

Trefzer

Quellenangaben

C. Mohan, "Aries: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging," Proceedings of the sixteenth ACM symposium on Operating systems principles, 1997
Andreas Reuter, "Concurrency Control in Database Systems," ACM Computing Surveys, 1983.
Jim Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger, "Granularity of Locks and Degrees of Consistency in a Shared Data Base," IFIP Working Conference on Modelling in Data Base Management Systems, 1976.

Stefan Lauber

Thorsten Kansy, www.dotnetconsulting.eu

http://www.imn.htwk-leipzig.de/~kudrass/Lehrmaterial/DB2-Impl/DB2-15/MSSQLServerDB-Glueck.pdf

SQL DATENBANKEN
TRANSAKTIONEN, ACID, CAP

Trefzer

Was lernen wir hier?

- Begrifflichkeiten Transaktion innerhalb der relationalen Datenbanken
- Was ist eine Transaktion
- Was versteht man unter Transaktionen innerhalb einer Datenbank und warum sind diese so wichtig!
- Explizite & Implizite Transaktionen
- Beispiele für Explizite & Implizite Transaktionen
- Das Konzept ACID
- Was steht hinter dem Konzept „CAP-Theorem“

SQL DATENBANKEN
TRANSAKTIONEN, ACID, CAP

Trefzer

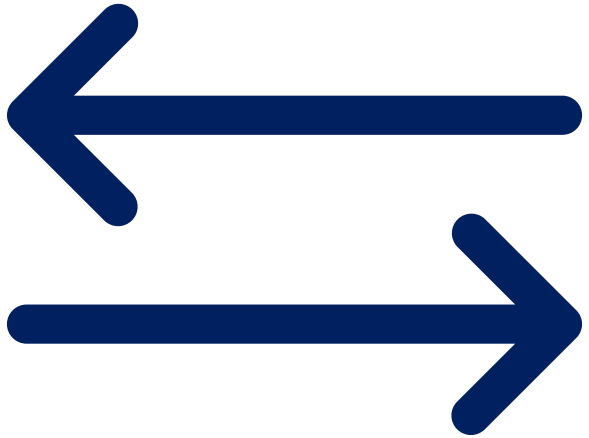
Extrem wichtige Funktionen einer relationalen Datenbank

- Reale Welt z. B. Essen gegen Geld…
- Transfer Betrag von Konto A nach Konto B = 2 Vorgänge
- Konto A: 5000 – 2000 = 3000
- Konto B: 1000 + 3000 = „ist gesperrt“
- Ergebnis: der Vorgang muss rückgängig gemacht werden
- Wichtige Funktion innerhalb der relationalen Datenbanken

Trefzer

Was ist eine Transaktion

- Eine Transaktion in relationalen Datenbanken ist eine Sequenz von Datenbankoperationen, die als eine einzelne und atomare Einheit behandelt werden.
- Das bedeutet, dass entweder alle Operationen erfolgreich abgeschlossen werden oder keine davon, um die Konsistenz der Datenbank sicherzustellen.
- Transaktionen folgen dem ACID-Prinzip, was für Atomicity (Atomarität), Consistency (Konsistenz), Isolation (Isolation) und Durability (Dauerhaftigkeit) steht.
- Dies stellt sicher, dass die Datenbank in einem konsistenten und zuverlässigen Zustand bleibt, selbst wenn es zu unerwarteten Ereignissen wie Systemabstürzen kommt.



Trefzer

ACID

- Atomicity (Atomarität):
  Eine Transaktion wird als eine unteilbare Einheit behandelt.
  Entweder werden alle Teiloperationen erfolgreich abgeschlossen oder keine davon. Sie folgt dem Prinzip „alles oder nichts“
- Consistency (Konsistenz):
  Die Datenbank muss von einem konsistenten Zustand in einen anderen konsistenten Zustand übergehen.
  Alle Integritätsregeln und Geschäftsregeln müssen eingehalten werden.

Trefzer

ACID

- Isolation (Isolation):
  Transaktionen laufen so, als ob sie einzeln ausgeführt werden, unabhängig von anderen Transaktionen.
  Verhindert unerwünschte Nebeneffekte und Konflikte zwischen Transaktionen.
- Durability (Dauerhaftigkeit):
  Einmal abgeschlossene Transaktionen werden permanent in der Datenbank gespeichert und überleben Systemabstürze oder Ausfälle.
  Die Daten bleiben auch nach einem Neustart der Datenbank erhalten.

Trefzer

Beispiel einer Transaktion

Ist eine Bündelung mehrere Operationen zu einer logischen Einheit

Konto A
Stand: 1000 €
Überweisung auf Konto B 200 €

Konto B
2000 € + 200 € = 2200 €

Transaktion:

1. lese Kontostand von A in _ka
2. Berechne _ka = _ka – 200
3. Schreibe neuen Kontostand _ka in A

4. Lese Kontostand von B in _kb
5. Berechne _kb = _kb + 200
6. Schreibe neuen Kontostand _kb in B

Trefzer

Rollback einer Transaktion

Es kommt zum Absturz der Datenbank

Konto A
1000 €

Rollback die Überweisung wird rückgängig gemacht!!

Konto B
2000 €

1. Berechne _ka = _ka – 200

Transaktion:

1. lese Kontostand von A in _ka
2. Schreibe neuen Kontostand _ka in A

Absturz!!

1. Lese Kontostand von B in _kb
2. Berechne _kb = _kb + 200
3. Schreibe neuen Kontostand _kb in B

Trefzer

Rollback einer Transaktion

- **Atomarität (Atomicity)** = Alle Aktionen der Transaktion werden ausgeführt oder keine d. h. alles oder nichts.
- **Konsistenz (Consistency)** = Die Transaktion muss die Datenbank in einem konsistenten Zustand hinterlassen. Werden Integritiätsbedingungen verletzt, werden die Daten zurückgeschrieben (Rollback)
- **Isolation (Isolation)** = Nebenläufige Transaktionen dürfen sich (logisch) nicht beeinflussen
- **Dauerhaftigkeit (Durability)** = Nur erfolgreiche abgeschlossene Transaktionen verbleiben in der Datenbank.

ACID ist ein Akronym, das die vier grundlegenden Eigenschaften von Transaktionen in relationalen Datenbanken beschreibt. ACID gewährleistet eine Ordnungsgemäßes funktionieren der Datenbank.

* Akronym = Abkürzung "NASA"

Trefzer

Explizite Transaktionen (Benutzer gesteuert)

Bei expliziten Transaktionen wird die Transaktion vom Benutzer oder dem Programmierer ausdrücklich gesteuert.

- Der Benutzer beginnt die Transaktion mit einem Befehl wie "BEGIN TRANSACTION" oder "START TRANSACTION" und beendet sie mit "COMMIT" oder "ROLLBACK".
- Alle Änderungen an der Datenbank, die während der Transaktion vorgenommen werden, sind vorübergehend und werden erst nach einem erfolgreichen "COMMIT" dauerhaft in der Datenbank gespeichert.
- Wenn etwas schief geht oder die Benutzerin oder der Benutzer die Transaktion abbrechen möchte, kann eine "ROLLBACK"-Anweisung verwendet werden, um alle vorgenommenen Änderungen rückgängig zu machen.

Trefzer

Explizite Transaktionen (Benutzer gesteuert)

Bei expliziten Transaktionen wird die Transaktion vom Benutzer oder dem Programmierer ausdrücklich gesteuert.

- Der Benutzer beginnt die Transaktion mit einem Befehl wie "BEGIN TRANSACTION" oder "START TRANSACTION" und beendet sie mit "COMMIT" oder "ROLLBACK".
- Alle Änderungen an der Datenbank, die während der Transaktion vorgenommen werden, sind vorübergehend und werden erst nach einem erfolgreichen "COMMIT" dauerhaft in der Datenbank gespeichert.
- Wenn etwas schief geht oder die Benutzerin oder der Benutzer die Transaktion abbrechen möchte, kann eine "ROLLBACK"-Anweisung verwendet werden, um alle vorgenommenen Änderungen rückgängig zu machen.

Trefzer

Beispiel: Explizite Transaktionen (Benutzer gesteuert)

Bei diesem Beispiel starten Sie mit „Beginn Transaction“, danach werden die Daten mittels Delete Anweisung gelöscht. Mit Rollback Transaction schreiben Sie die Daten zurück. Mit Commit Transaktionen werden die Daten festgeschrieben. Bei diesem Beispiel werden die Daten endgültig gelöscht.

```
-- Explizite Transaktionen ( muss vorher ausgeführt werden )

Begin Transaction

DELETE FROM dbo.Test;

-- mit Rollback Transaction werden die Daten zurückgeschrieben
Rollback Transaction

-- mit Commit Transaction werden die Daten endgültig gelöscht
Commit Transaction

-- Währende Transatkonen laufen, sind diese Datensätze in einer Mehrbenutzer Umgebung gesperrt.
```

Trefzer

Implizite Transaktionen (Standard)

- Im Gegensatz dazu erfolgen implizite Transaktionen automatisch, ohne dass der Benutzer sie ausdrücklich startet oder beendet.
- Jede einzelne Datenbankoperation (z. B. SELECT, INSERT, UPDATE, DELETE) wird als separate Transaktion behandelt.
- Die Änderungen an der Datenbank werden sofort und implizit gespeichert, sobald die jeweilige Operation erfolgreich abgeschlossen ist.
- Es gibt keine Möglichkeit, die Änderungen mehrerer Operationen als Einheit zu behandeln oder rückgängig zu machen, es sei denn, es wird auf explizite Transaktionen umgestellt.

Trefzer

Beispiel: Implizite Transaktionen (Standard)

Hier sehen Sie ein Beispiel einer implizierten Transaktion. Der Löschvorgang wird abgebrochen, da zur Tabelle Ausschuss referentielle Integrität besteht. Es findet ein automatisches Rollback statt. Die Daten werden nicht verändert. Hier gilt wieder immer **alles oder nichts**!

SRV-DB.Produktion - dbo.Arbeiter | Transaktion.sql - S...Produktion (sa (51))* | SRV-DB.Produktion...uktion - Diagram_0

```sql
use Produktion

-- Transaktionen

-- Implizite Transaktionen (kann nicht löschen da referentielle Integrität besteht)
-- bei der Ausführung kommte es zu einem Fehler

DELETE FROM Arbeiter;

-- die Tabelle Arbeiter steht mit der Tabelle Ausschuss in Verbindung.
```

SQL DATENBANKEN
TRANSAKTIONEN, ACID, CAP

Trefzer

Warum?

- In der Regel werden explizite Transaktionen verwendet, wenn komplexe Aktionen in einer Datenbank ausgeführt werden müssen, die als Einheit behandelt werden sollen.

- Implizite Transaktionen werden in einfachen Fällen verwendet, in denen die Datenkonsistenz weniger kritisch ist und die Performance eine größere Rolle spielt.

Trefzer

CAP-Theorem

- Grundlegendes Konzept innerhalb der DB-Theorie
- Beschreibt die Kompromisse zwischen drei wichtigen Eigenschaften
- Die in verteilten Datenbanken auftreten können

C steht für **Konsistenz (Consistency)**
A steht für **Verfügbarkeit (Availability)**
P steht für **Partitionstoleranz (Partition Tolerance)**

Trefzer

CAP-Theorem

Konsistenz: Dies bedeutet, dass alle Knoten in einem verteilten System zu jedem Zeitpunkt die gleichen Daten anzeigen. Mit anderen Worten, wenn Sie eine Änderung an den Daten vornehmen, wird diese Änderung sofort für alle Knoten im System sichtbar.

Verfügbarkeit: Bedeutet, dass jeder Lese- oder Schreibzugriff auf das System eine Antwort erhalten kann, ohne dass es zu Ausfällen kommt. Das System ist ständig erreichbar und gibt immer eine Antwort zurück, auch wenn die Antwort möglicherweise nicht die aktuellsten Daten enthält.

Partitionstoleranz: Partitionstoleranz bezieht sich darauf, wie das System mit Netzwerkausfällen oder Verbindungsproblemen zwischen den Knoten umgeht. Ein verteiltes System ist partitionstolerant, wenn es auch dann weiterhin funktioniert, wenn die Kommunikation zwischen den Knoten unterbrochen ist.

Trefzer

CAP-Theorem

Das CAP-Theorem besagt, dass in einem verteilten Datenbanksystem gleichzeitig nur zwei der oben genannten Eigenschaften maximal erreicht werden können.

Das bedeutet, dass, wenn Sie eine hochkonsistente (DB-C) Datenbank haben möchten, die immer aktuelle Daten liefert, sie möglicherweise nicht immer verfügbar (DB-A) ist, insbesondere bei Netzwerkausfällen.

Umgekehrt, wenn Sie eine hochverfügbare (DB-A) Datenbank wünschen, die immer erreichbar ist, könnten Sie Kompromisse bei der Konsistenz (DB-C) eingehen müssen, was bedeutet, dass einige Knoten möglicherweise nicht die aktuellsten Daten anzeigen.

Trefzer

CAP-Theorem

Partitionstoleranz (P)
ist in der Regel ein unverzichtbares Merkmal, da verteilte Systeme mit Netzwerkausfällen umgehen müssen. Daher müssen Sie immer zwischen Konsistenz (C) und Verfügbarkeit (A) wählen, wenn es zu Netzwerkpartitionen kommt.

Zusammengefasst bedeutet das CAP-Theorem,
dass Sie in einem verteilten System Kompromisse zwischen Konsistenz und Verfügbarkeit eingehen müssen, abhängig von den Anforderungen Ihres Systems und den Bedingungen, unter denen es betrieben wird. Es gibt verschiedene Datenbanken und Datenbankmodelle, die diese Kompromisse auf unterschiedliche Weisen behandeln, je nach den spezifischen Anforderungen Ihres Projekts.

VIELEN DANK

MARKUS TREFZER